

## Chapitre 1 : Introduction à l'administration d'hébergement web

### I. Principes fondamentaux de l'hébergement web

L'hébergement web est un service essentiel qui permet de rendre des sites web et des applications accessibles sur Internet. L'hébergement web consiste à stocker les fichiers d'un site web (HTML, CSS, JavaScript, bases de données, images, etc.) sur un serveur connecté à Internet, permettant aux utilisateurs d'y accéder via un navigateur. Le serveur est une machine physique ou virtuelle qui héberge ces données et répond aux requêtes HTTP/HTTPS.

#### I.1. Les composants essentiels

**Le serveur :** C'est un ordinateur puissant qui stocke vos fichiers et les délivre aux visiteurs qui accèdent à votre site. Il fonctionne 24h/24 et 7j/7.

**Le nom de domaine :** C'est l'adresse de votre site (exemple.com). Il est lié à l'adresse IP du serveur via le système DNS (Domain Name System) qui traduit les noms de domaine en adresses IP.

**La bande passante :** C'est la quantité de données qui peut être transférée entre votre site et ses visiteurs. Plus vous avez de trafic, plus vous consommez de bande passante.

**L'espace de stockage :** La quantité de données que vous pouvez stocker sur le serveur (fichiers du site, bases de données, emails, etc.).

**La base de données:**

**Rôle :** Stockage structuré (MySQL, PostgreSQL) pour données dynamiques.

**Exemple :** Une boutique en ligne stocke produits et commandes dans MySQL.

**Le panneau de contrôle :**

**Outils :** cPanel, Plesk, pour gérer domaines, emails, bases de données.

**Exemple :** cPanel permet de créer une adresse [email@exemple.com](mailto:email@exemple.com).

## II. Typologie des solutions d'hébergement (mutualisé, VPS, dédié, cloud)

Les types d'hébergement varient selon les besoins en ressources, contrôle et budget.

### II.1. Hébergement mutualisé

Plusieurs sites partagent un même serveur et ses ressources (CPU, RAM).

**Avantages :** Abordable (5-15 €/mois), facile à gérer (maintenance par le fournisseur).

**Inconvénients :** Performances limitées si un site consomme trop de ressources.

**Exemple :** OVH Personal Hosting, idéal pour blogs ou sites vitrines.

### II.2. Hébergement VPS (Serveur Privé Virtuel)

Un serveur physique divisé en serveurs virtuels isolés, offrant des ressources dédiées.

**Avantages :** Plus de contrôle (accès root), meilleure performance.

**Inconvénients :** Coût plus élevé (20-100 €/mois), nécessite compétences d'administration.

**Exemple :** DigitalOcean Droplets pour développeurs.

### II.3. Hébergement dédié

Un serveur physique entier dédié à un seul client.

**Avantages :** Performances maximales, personnalisation complète.

**Inconvénients :** Très coûteux (100-500 €/mois), gestion complexe.

**Exemple :** Serveurs dédiés OVH pour grandes entreprises.

# Administration Hébergement Web

## II.4. Hébergement cloud

Ressources réparties sur plusieurs serveurs virtuels dans le cloud, évolutives.

Avantages : Scalabilité (ajustement automatique), résilience.

Inconvénients : Coût variable, complexité de gestion.

Exemple : AWS EC2, Google Cloud, Azure.

## II.5. Hébergement spécialisé

Exemples : WordPress (optimisé pour CMS), e-commerce (Magento).

Avantages : Préconfiguré pour des plateformes spécifiques.

Inconvénients : Moins flexible pour d'autres usages.

Tableau comparatif :

| Type      | Coût (€/mois) | Performance | Contrôle | Cas d'usage  |
|-----------|---------------|-------------|----------|--------------|
| Mutualisé | 5-15          | Faible      | Limité   | Blogs        |
| VPS       | 20-100        | Moyenne     | Élevé    | Sites moyens |
| Dédié     | 100-500       | Élevée      | Total    | Grandes apps |
| Cloud     | Variable      | Évolutive   | Variable | Scalabilité  |

## II.6. Critères de Choix d'un Hébergement Web

### II.6.1. 5.1 Besoins du site

**Trafic** : Mutualisé pour <10 000 visites/mois, VPS/dédié pour plus.

**Type de contenu** : Statique (HTML) ou dynamique (CMS, base de données).

**Exemple** : Un blog personnel peut utiliser mutualisé ; une boutique e-commerce préfère VPS/cloud.

# Administration Hébergement Web

## **II.6.2. 5.2 Performance**

**Temps de chargement** : SSD et CDN (Content Delivery Network) réduisent la latence.

**Uptime** : Garantie >99,9% (ex. : AWS promet 99,99%).

## **II.6.3. 5.3 Budget**

Mutualisé pour petits budgets ; cloud pour évolutivité.

## **II.6.4. 5.4 Support technique**

24/7 via chat/ticket, crucial pour entreprises.

## **II.6.5. 5.5 Scalabilité**

Cloud/VPS pour ajout rapide de ressources.

## **III. Enjeux de sécurité et de confidentialité des données**

Les enjeux de sécurité et de confidentialité des données dans l'administration et l'hébergement web sont cruciaux à l'ère numérique, particulièrement en 2025, où la dépendance aux services en ligne et les cyberattaques sont en constante augmentation. Ces enjeux touchent à la protection des données des utilisateurs, la résilience des systèmes face aux menaces, et le respect des réglementations internationales.

### **III.1.1. 1. Contexte et Importance**

#### **III.1.1.1. 1.1 Pourquoi la sécurité et la confidentialité sont-elles critiques ?**

**Données sensibles** : Les sites web, qu'il s'agisse de boutiques e-commerce, de blogs ou d'applications SaaS, stockent des informations sensibles (noms, adresses, numéros de carte bancaire, données médicales).

# Administration Hébergement Web

**Impact des violations** : Une fuite de données peut entraîner des pertes financières, une atteinte à la réputation, et des sanctions légales (ex. : amendes RGPD pouvant atteindre 20M€ ou 4% du chiffre d'affaires annuel mondial).

**Évolution des menaces** : En 2025, les attaques par ransomware, phishing, et DDoS sont sophistiquées, avec une augmentation de 30% des cyberattaques par rapport à 2023 (source : rapports Cybersecurity Ventures).

**Exemple** : En 2024, une fuite de données chez un hébergeur européen a exposé 10 millions de dossiers clients, soulignant l'importance de la sécurité.

## III.1.1.2. 1.2 Enjeux spécifiques à l'hébergement web

**Accessibilité continue** : Les serveurs doivent rester disponibles malgré les attaques (ex. : DDoS).

**Protection des données en transit** : Les échanges entre client et serveur doivent être sécurisés.

**Conformité légale** : Respecter des lois comme le RGPD (Europe) ou le CCPA (Californie).

**Responsabilité partagée** : Dans le cloud (ex. : AWS), l'hébergeur sécurise l'infrastructure, mais le client configure la sécurité logicielle.

## III.1.2. 2. Principaux Enjeux de Sécurité

### III.1.2.1. 2.1 Attaques courantes

**Attaques DDoS (Distributed Denial of Service)** : Surcharge du serveur par des requêtes massives, rendant le site inaccessible. Ex. : En 2023, Cloudflare a bloqué une attaque DDoS de 71M requêtes/seconde.

**SQL Injection** : Insertion de code malveillant dans une requête SQL pour accéder à une base de données. Ex. : Vol de mots de passe via un formulaire non sécurisé.

**Cross-Site Scripting (XSS)** : Injection de scripts dans les pages web pour voler des cookies ou rediriger les utilisateurs.

# Administration Hébergement Web

**Ransomware** : Chiffrement des données du serveur, exigeant une rançon. Ex. : Attaque contre un hébergeur en 2024, paralysant 50 sites.

**Phishing** : Emails frauduleux imitant des services légitimes pour voler des identifiants.

## III.1.2.2. 2.2 Vulnérabilités des serveurs

**Logiciels non mis à jour** : Les CMS comme WordPress (45% des sites web en 2025) ou les serveurs web (Apache, Nginx) sont vulnérables si non patchés. Ex. : Une faille WordPress 6.6 exploitée en 2024.

**Mauvaise configuration** : Ports ouverts inutiles (ex. : port 22 SSH non sécurisé) ou permissions excessives.

**Faibles authentifications** : Mots de passe faibles ou absence de 2FA (authentification à deux facteurs).

## III.1.2.3. 2.3 Menaces internes

**Erreurs humaines** : Un administrateur peut exposer des clés API par erreur (ex. : clé AWS sur GitHub).

**Mauvaise gestion des accès** : Trop de privilèges accordés à un utilisateur ou employé.

## III.1.3. 3. Enjeux de Confidentialité des Données

### III.1.3.1. 3.1 Protection des données personnelles

**Définition** : Les données personnelles incluent noms, emails, adresses, données biométriques, etc., protégées par des lois comme le RGPD.

**Défi** : Garantir que seules les données nécessaires sont collectées (principe de minimisation) et qu'elles sont stockées de manière sécurisée.

**Exemple** : Un site e-commerce doit chiffrer les informations de paiement et ne pas stocker les numéros de carte bancaire inutilement.

# Administration Hébergement Web

## III.1.3.2. 3.2 Conformité légale

**RGPD (2018, mis à jour 2024)** : Exige le consentement explicite pour collecter des données, droit à l'oubli, et notification des violations sous 72h.

**CCPA (Californie)** : Droit des utilisateurs à savoir quelles données sont collectées et à demander leur suppression.

**Autres** : Lois similaires au Canada (PIPEDA) ou au Japon (APPI).

**Exemple** : Un hébergeur basé à Roubaix (OVH) doit garantir que les données des clients européens restent dans l'UE.

## III.1.3.3. 3.3 Transparence et consentement

**Défi** : Informer les utilisateurs sur l'usage de leurs données (ex. : cookies).

**Solution** : Bandeaux de consentement conformes RGPD, politiques de confidentialité claires.

## III.1.4. 4. Bonnes Pratiques pour la Sécurité et la Confidentialité

### III.1.4.1. 4.1 Sécurisation du serveur

**Mises à jour régulières** : Patcher OS (ex. : Ubuntu 24.04), serveurs web (Apache 2.4.62), et CMS.

**Pare-feu** : Configurer UFW (Linux) ou pare-feu cloud (AWS WAF) pour filtrer le trafic.

**SSL/TLS** : Activer HTTPS avec certificats gratuits via Let's Encrypt ou payants (ex. : DigiCert). Ex. : certbot --apache pour installer un certificat.

**Protection DDoS** : Utiliser des services comme Cloudflare ou AWS Shield.

**Chiffrement** : Chiffrer les bases de données (ex. : MySQL avec AES-256).

### III.1.4.2. 4.2 Gestion des accès

**Authentification forte** : Activer 2FA pour cPanel, SSH, ou accès cloud.

# Administration Hébergement Web

**Principes de moindre privilège** : Limiter les droits (ex. : utilisateur MySQL avec accès lecture seule).

**Audit des accès** : Surveiller les connexions (ex. : journaux SSH via /var/log/auth.log).

## III.1.4.3. 4.3 Sauvegardes et récupération

**Fréquence** : Sauvegardes quotidiennes/hebdomadaires, stockées hors site (ex. : AWS S3).

**Restauration** : Tester régulièrement les backups. Ex. : Restaurer une base MySQL avec `mysql -u root -p < backup.sql`.

**Exemple** : Un hébergeur comme SiteGround propose des backups automatiques avec restauration en 1 clic.

## III.1.4.4. 4.4 Sécurisation des applications

**Validation des entrées** : Utiliser des fonctions comme `filter_var()` en PHP pour bloquer les injections.

**Protection XSS** : Échapper les sorties (ex. : `htmlspecialchars()`).

**Frameworks sécurisés** : Laravel ou Django intègrent des protections CSRF/XSS.

## III.1.4.5. 4.5 Conformité et confidentialité

**RGDPD Compliance** : Héberger en UE pour clients européens, utiliser des clauses contractuelles types (CCT) pour transferts hors UE.

**Chiffrement des données** : En transit (TLS) et au repos (AES).

**Audit régulier** : Scanner les vulnérabilités avec outils comme Nessus ou OpenVAS.

## IV. Rôles, responsabilités et compétences de l'administrateur système/web

L'administrateur système/web joue un rôle central dans l'hébergement web et la gestion des infrastructures informatiques. En 2025, avec l'essor du cloud computing, des

# Administration Hébergement Web

applications web complexes et des cybermenaces sophistiquées, ce rôle exige une combinaison de compétences techniques, analytiques et organisationnelles.

## IV.1.1. 2. Rôles de l'Administrateur Système/Web

Les rôles varient selon la taille de l'entreprise, le type d'hébergement (mutualisé, VPS, cloud) et le contexte (PME, grande entreprise, fournisseur d'hébergement). Voici les principaux rôles :

### IV.1.1.1. 2.1 Gestion des serveurs

**Configuration** : Installation et configuration des serveurs web (Apache, Nginx), bases de données (MySQL, PostgreSQL) et systèmes d'exploitation (Linux, Windows Server).

**Surveillance** : Suivi des performances (CPU, RAM, disque) et détection des pannes ou goulots d'étranglement.

**Exemple** : Configurer un VPS Ubuntu avec `sudo apt install nginx` et surveiller via `htop`.

### IV.1.1.2. 2.2 Administration des services web

**Serveurs web** : Gestion des fichiers de configuration (ex. : `/etc/nginx/nginx.conf`) pour optimiser les performances.

**Bases de données** : Création, maintenance et optimisation (ex. : indexation MySQL).

**Exemple** : Optimiser une base de données WordPress en supprimant les entrées inutiles avec phpMyAdmin.

### IV.1.1.3. 2.3 Sécurité

**Protection** : Mise en place de pare-feu (UFW, iptables), certificats SSL (Let's Encrypt), et protection contre DDoS (Cloudflare).

**Réponse aux incidents** : Analyse des logs (`/var/log/apache2/error.log`) pour identifier une attaque XSS.

**Exemple** : Configurer un WAF (Web Application Firewall) sur AWS pour bloquer les requêtes malveillantes.

# Administration Hébergement Web

## IV.1.1.4. 2.4 Gestion des domaines et DNS

**Configuration DNS** : Gestion des enregistrements (A, CNAME, MX) pour lier domaines à serveurs.

**Exemple** : Ajouter un enregistrement A dans Cloudflare pour pointer `www.exemple.com` vers une IP serveur.

## IV.1.1.5. 2.5 Sauvegardes et récupération

**Planification** : Mise en place de sauvegardes automatiques (ex. : rsync vers un stockage S3).

**Restauration** : Tester la récupération (ex. : restaurer une base MySQL avec `mysql -u root -p < backup.sql`).

**Exemple** : Restaurer un site après une attaque ransomware grâce à un backup hebdomadaire.

## IV.1.1.6. 2.6 Support technique

**Assistance utilisateurs** : Résoudre les problèmes (ex. : erreurs 500, emails non fonctionnels).

**Exemple** : Aider un client à configurer une adresse email via cPanel.

## IV.1.1.7. 2.7 Automatisation et scripting

**Scripts** : Automatiser les tâches (ex. : script Bash pour mises à jour serveur).

**Exemple** : Utiliser Ansible pour déployer des configurations sur plusieurs serveurs.

## IV.1.2. 3. Responsabilités de l'Administrateur Système/Web

Les responsabilités sont étroitement liées aux rôles, mais elles se concentrent sur les résultats attendus et les obligations.

### IV.1.2.1. 3.1 Garantir la disponibilité (Uptime)

**Objectif** : Maintenir un uptime >99,9% (standard industriel).

# Administration Hébergement Web

**Tâches** : Surveiller avec des outils comme Nagios ou Zabbix, redémarrer les services si nécessaire.

**Exemple** : Résoudre une panne serveur en redémarrant Nginx (`sudo systemctl restart nginx`).

## IV.1.2.2. 3.2 Optimiser les performances

**Tâches** : Optimiser la configuration du serveur web (ex. : activer la compression Gzip dans Nginx), utiliser des CDN, ajuster les paramètres de base de données.

**Exemple** : Réduire le temps de chargement d'un site WordPress de 3s à 1s en activant un cache (W3 Total Cache).

## IV.1.2.3. 3.3 Assurer la sécurité

**Tâches** : Appliquer des correctifs de sécurité, configurer 2FA, surveiller les vulnérabilités (ex. : avec OpenVAS).

**Exemple** : Bloquer une attaque DDoS en 2025 en activant Cloudflare Proxy.

## IV.1.2.4. 3.4 Conformité légale

**Responsabilité** : Respecter des réglementations comme le RGPD (stockage des données dans l'UE, consentement cookies).

**Exemple** : Configurer un bandeau de consentement RGPD-compliant sur un site.

## IV.1.2.5. 3.5 Gestion des ressources

**Tâches** : Allouer CPU/RAM en fonction des besoins, gérer les quotas (ex. : limiter l'espace disque dans un hébergement mutualisé).

**Exemple** : Augmenter la RAM d'un VPS AWS EC2 pour supporter un pic de trafic.

## IV.1.2.6. 3.6 Documentation et rapports

**Tâches** : Documenter les configurations, incidents, et procédures de récupération.

**Exemple** : Rédiger un rapport après une attaque SQL injection pour éviter sa répétition.

# Administration Hébergement Web

## IV.1.3. 4. Compétences Requises

Les compétences d'un administrateur système/web sont techniques, mais aussi transversales, pour répondre aux défis modernes.

### IV.1.3.1. 4.1 Compétences techniques

**Systèmes d'exploitation** : Maîtrise de Linux (Ubuntu, CentOS) et Windows Server. Ex. : Commandes comme ls, chmod, systemctl.

**Serveurs web** : Configuration d'Apache, Nginx, gestion de fichiers .htaccess.

**Bases de données** : Administration de MySQL, PostgreSQL (ex. : optimiser avec EXPLAIN en SQL).

**Réseaux** : Connaissance des protocoles (HTTP, DNS, FTP, SSH), configuration de ports.

**Scripting** : Bash, Python, ou PowerShell pour automatisation (ex. : script pour nettoyer les logs).

**Cloud** : Familiarité avec AWS, Azure, Google Cloud (ex. : gérer des instances EC2).

**Sécurité** : Gestion des certificats SSL, pare-feu, détection d'intrusions.

### IV.1.3.2. 4.2 Compétences en sécurité

**Analyse des menaces** : Identifier les vulnérabilités avec outils comme Nessus.

**Réponse aux incidents** : Suivre un plan (ex. : isoler un serveur compromis).

**Conformité** : Comprendre RGPD, CCPA pour configurer les systèmes.

### IV.1.3.3. 4.3 Compétences transversales

**Résolution de problèmes** : Diagnostiquer une erreur 500 via les logs.

**Communication** : Expliquer des problèmes techniques aux clients non techniques.

**Gestion du temps** : Prioriser les tâches en cas de panne critique.

**Apprentissage continu** : Suivre les évolutions (ex. : nouvelles versions d'Ubuntu ou failles CVE).

# Administration Hébergement Web

## V. Panorama des outils d'administration d'hébergement (cPanel, SSH, Webmin, etc.)

Les outils d'administration d'hébergement web sont essentiels pour gérer, surveiller et sécuriser les serveurs et les services web, que ce soit pour des sites statiques, des applications dynamiques ou des infrastructures cloud. En 2025, ces outils se diversifient pour répondre à la complexité croissante des environnements d'hébergement (mutualisé, VPS, dédié, cloud).

### V.1. 1. Introduction aux Outils d'Administration

Les outils d'administration d'hébergement web simplifient les tâches complexes comme la configuration des serveurs, la gestion des bases de données, la surveillance des performances, la sécurité, et les sauvegardes. Ils se divisent en plusieurs catégories : panneaux de contrôle, outils de ligne de commande (CLI), outils de surveillance, solutions de sécurité, et plateformes d'automatisation. Chaque outil répond à des besoins spécifiques, du débutant gérant un site WordPress au professionnel administrant une flotte de serveurs cloud.

**Objectif :** Permettre une gestion efficace, sécurisée et automatisée des infrastructures web tout en minimisant les erreurs humaines.

**Contexte 2025 :** Avec l'essor du cloud (AWS, Azure) et des CMS (WordPress représentant 45% des sites), les outils doivent être polyvalents, scalables et conformes aux réglementations comme le RGPD.

### V.2. 2. Catégories d'Outils d'Administration

#### V.2.1. 2.1 Panneaux de Contrôle

Les panneaux de contrôle offrent une interface graphique (GUI) pour gérer les aspects de l'hébergement sans expertise CLI.

- **cPanel :**

# Administration Hébergement Web

**Fonctionnalités** : Gestion de domaines, emails, bases de données MySQL, fichiers, SSL, sauvegardes.

**Cas d'usage** : Hébergement mutualisé/VPS (ex. : OVH, SiteGround).

**Avantages** : Interface intuitive, support WordPress, intégration Let's Encrypt.

**Inconvénients** : Coût de licence (15-50 €/mois), limité pour serveurs dédiés complexes.

**Exemple** : Créer une adresse [email@domaine.com](mailto:email@domaine.com) en 2 clics.

- **Plesk** :

**Fonctionnalités** : Similaire à cPanel, mais multi-OS (Linux/Windows), gestion Docker.

**Cas d'usage** : Hébergement multi-plateforme, PME.

**Avantages** : Support Windows Server, extensions (ex. : WordPress Toolkit).

**Inconvénients** : Courbe d'apprentissage plus raide que cPanel.

**Exemple** : Configurer un serveur Node.js via Plesk.

- **Webmin** :

**Fonctionnalités** : Open source, gestion utilisateurs, serveurs web (Apache/Nginx), SSH.

**Cas d'usage** : Serveurs Linux pour administrateurs expérimentés.

**Avantages** : Gratuit, personnalisable.

**Inconvénients** : Interface moins moderne.

**Exemple** : Gérer les permissions Linux via interface graphique.

## V.2.2. 2.2 Outils de Ligne de Commande (CLI)

Les outils CLI sont privilégiés pour leur précision et leur flexibilité, particulièrement sur Linux.

# Administration Hébergement Web

- **SSH (Secure Shell) :**
  - **Fonctionnalités :** Accès distant sécurisé au serveur (ex. : `ssh user@ip`).
  - **Cas d'usage :** Configuration manuelle, debugging.
  - **Exemple :** Redémarrer Nginx avec `sudo systemctl restart nginx`.
- **rsync :**
  - **Fonctionnalités :** Synchronisation/sauvegarde de fichiers (ex. : `rsync -av /site /backup`).
  - **Cas d'usage :** Sauvegardes locales ou distantes.
  - **Exemple :** Sauvegarder un site vers AWS S3.
- **htop :**
  - **Fonctionnalités :** Surveillance interactive des processus (CPU, RAM).
  - **Cas d'usage :** Identifier un processus gourmand.
  - **Exemple :** Tuer un processus PHP bloqué.
- **Certbot :**
  - **Fonctionnalités :** Installation/gestion de certificats SSL Let's Encrypt (ex. : `certbot --nginx`).
  - **Cas d'usage :** Sécurisation HTTPS.
  - **Exemple :** Activer HTTPS pour un site en 5 minutes.

## V.2.3. 2.3 Outils de Surveillance

Ces outils surveillent les performances, la disponibilité et les anomalies.

- **Nagios :**
  - **Fonctionnalités :** Surveillance serveur (uptime, CPU, disque), alertes (email/Slack).
  - **Cas d'usage :** Serveurs dédiés/VPS.

# Administration Hébergement Web

- **Avantages** : Personnalisable, plugins nombreux.
- **Inconvénients** : Configuration complexe.
- **Exemple** : Alerte si uptime <99,9%.
- **Prometheus + Grafana** :
  - **Fonctionnalités** : Collecte de métriques (Prometheus), visualisation (Grafana).
  - **Cas d'usage** : Infrastructures cloud (AWS, Azure).
  - **Avantages** : Dashboards visuels, scalabilité.
  - **Exemple** : Graphique du trafic réseau en temps réel.
- **UptimeRobot** :
  - **Fonctionnalités** : Surveillance de sites web (ping toutes les 5 min).
  - **Cas d'usage** : Petits sites mutualisés.
  - **Avantages** : Gratuit (jusqu'à 50 moniteurs).
  - **Exemple** : Notification SMS si site hors ligne.

## V.2.4. 2.4 Outils de Sécurité

Ces outils protègent contre les cyberattaques et assurent la conformité.

- **Fail2ban** :
  - **Fonctionnalités** : Analyse des logs pour bloquer IP malveillantes (ex. : attaques SSH).
  - **Cas d'usage** : Serveurs Linux.
  - **Exemple** : Bloquer une IP après 5 tentatives de connexion échouées.
- **ModSecurity** :
  - **Fonctionnalités** : WAF (Web Application Firewall) pour bloquer SQL injection/XSS.

# Administration Hébergement Web

- **Cas d'usage** : Sites dynamiques (WordPress, Laravel).
- **Exemple** : Filtrer les requêtes HTTP malveillantes.
- **Cloudflare** :
  - **Fonctionnalités** : CDN, protection DDoS, gestion DNS.
  - **Cas d'usage** : Tous types de sites.
  - **Avantages** : Facile à configurer, plan gratuit.
  - **Exemple** : Mitiger une attaque DDoS en 2025.
- **OpenVAS** :
  - **Fonctionnalités** : Scanner de vulnérabilités.
  - **Cas d'usage** : Audit de sécurité.
  - **Exemple** : Détecter une version obsolète de PHP.

## V.2.5. 2.5 Outils d'Automatisation

L'automatisation réduit les tâches répétitives et améliore l'efficacité.

- **Ansible** :
  - **Fonctionnalités** : Automatisation des configurations via playbooks YAML.
  - **Cas d'usage** : Gestion multi-serveurs.
  - **Exemple** : Déployer Nginx sur 10 VPS en une commande.
- **Terraform** :
  - **Fonctionnalités** : Infrastructure as Code (IaC) pour provisionner serveurs cloud.
  - **Cas d'usage** : AWS, Azure, GCP.
  - **Exemple** : Créer une instance EC2 via un script.
- **Docker** :

# Administration Hébergement Web

- **Fonctionnalités** : Conteneurisation pour isoler applications.
- **Cas d'usage** : Déploiement d'applications (ex. : WordPress dans un conteneur).
- **Exemple** : Lancer un conteneur MySQL avec `docker run mysql`.